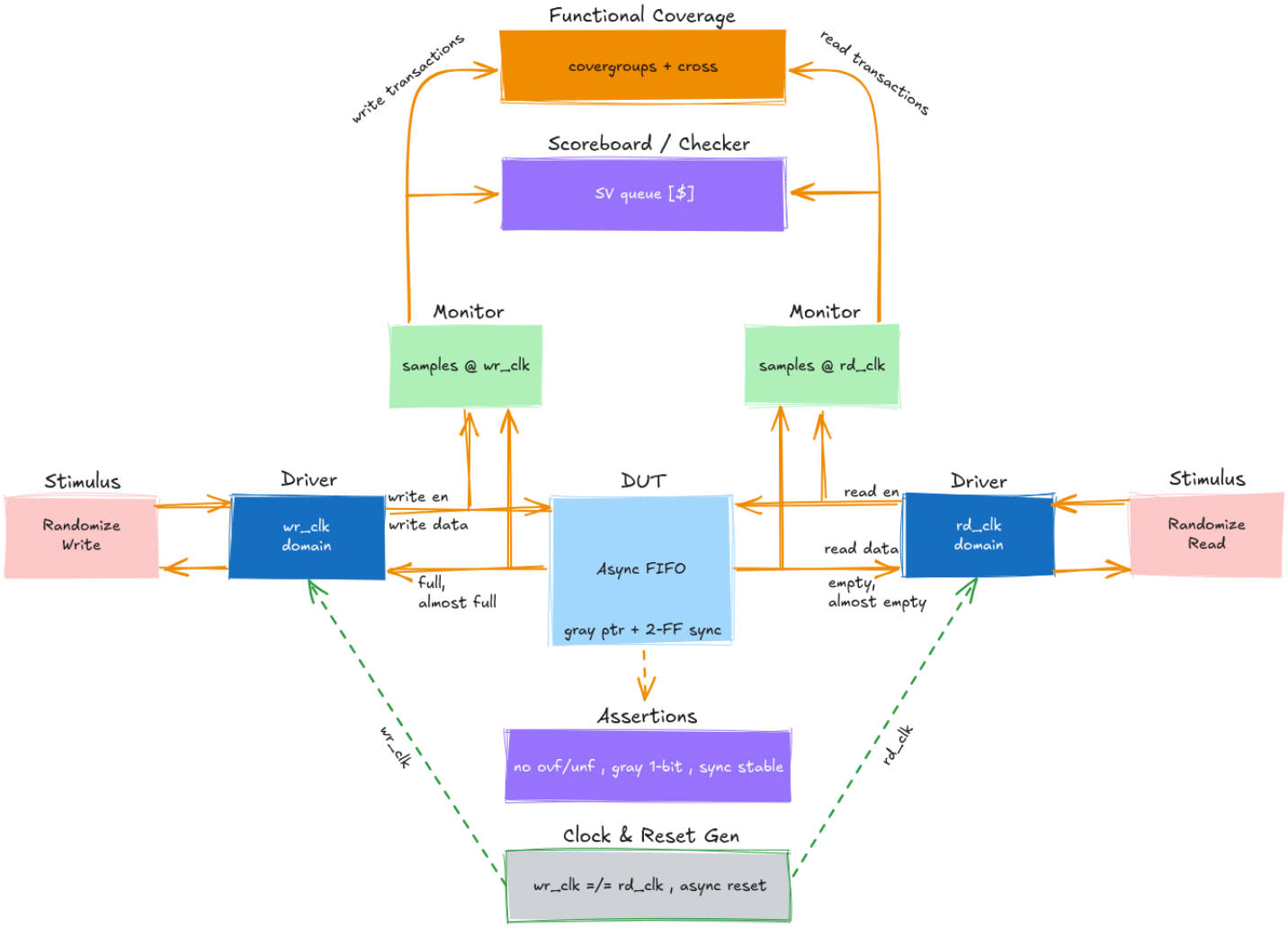
Report

ADDV / 11.xx.2026

Part 1:

1. Testbench architecture and strategy:



- **Simulation:** directed tests for basic operation, constrained-random tests across 10+ seeds, several clock ratios, FIFO depths and widths, plus targeted corner cases.
- **Formal:** run on a small configuration (4 entries x 2-bit data) so proofs converge:
  - Prove every assertion holds for all possible inputs.
  - Prove data comes out in the order it went in, by tracking one data word through the FIFO.
  - Prove the FIFO never locks up and can always reach full and empty.

- **Sign-off metrics**
  - **Functional coverage:** 100%, with any exclusion documented and justified.
  - **Code coverage**: ≥ 95%, with every remaining gap reviewed and waived.
  - **Assertions**: Each one triggered at least once, and none failed.
  - **Formal**: All properties proven, or bound to at least 2 × depth + synchronizer delay.
  - **Scoreboard**: Zero mismatches across the full regression.
  - **Regression**: 100% pass on every seed, clock ratio and configuration.
  - **Bugs**: No open critical or major bugs, and the rate of new bugs has levelled off.

2. Stimulus and coverage plan:

#	Scenario	Stimulus	Coverage
Directed	Reset	Assert both resets,then release them	Flags after reset: empty = 1, full = 0
	Single write then read	1 write, then 1 read	Basic data path
	Fill to full	Writes only, until full	Full reached
	Drain to empty	Reads only, until empty	Empty reached
	Overflow attempt	Keep writing while full	Write attempted while full
	Underflow attempt	Keep reading while empty	Read attempted while empty
Random	Random traffic	Random enables and data	Fill bins: 0, 1, mid, depth-1, depth
	Write faster than read	Write clock = 2× read clock	Clock-ratio bin
	Read faster than write	Write clock = 0.5× read clock	
	Unrelated clocks	Random periods and phase offsets	
Corner	Simultaneous R/W at the limits	Write and read in the same cycle while full, and while empty	R/W at full, R/W at empty
	Almost-full/ almost-empty thresholds	Fill and drain across each threshold	Each flag rising and falling
	Flag change at wrap-around	Fill or drain so a flag changes as a pointer wraps	Flag change × wrap
	Access right after reset	Write or read in the first cycle after reset is released	First access after reset
	Clock edges close together	Put write and read clock edges very close	Synchronizer stress

3. Checking plan:

Check	Where
Read data matches the expected data, in write order	**Scoreboard**
No data is lost or duplicated, and the model is empty at the end of the test	
Almost-full and almost-empty flags change exactly at their thresholds	
full / empty matches the model's count	**Assertion**
No write is accepted while full (no overflow)	
No read is accepted while empty (no underflow)	
After reset, empty = 1, full = 0 and the pointers are 0	
Gray-coded pointers change only one bit at a time	
No X or Z on outputs after reset	